

1. Handling Missing Data

IDLE Shell 3.11.9

File Edit Shell Debug Options Window Help

Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
= RESTART: C:/Users/btaru/Using head() and tail().py

First 5 Rows:

	Name	Score
0	Alice	85
1	Bob	92
2	Charlie	78
3	David	95
4	Eva	88

First 2 Rows:

	Name	Score
0	Alice	85
1	Bob	92

Last 3 Rows:

	Name	Score
4	Eva	88
5	Frank	72
6	Grace	91

>>>

Using head() and tail().py - C:/Users/btaru/Using head() and tail().py (3.11.9)

File Edit Format Run Options Window Help

```
import pandas as pd

# Sample data
data = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva", "Frank", "Grace"],
    "Score": [85, 92, 78, 95, 88, 72, 91]
}

# Create DataFrame
df = pd.DataFrame(data)

# Display first 5 rows
print("First 5 Rows:")
print(df.head())

# Display first 2 rows
print("\nFirst 2 Rows:")
print(df.head(2))

# Display last 3 rows
print("\nLast 3 Rows:")
print(df.tail(3))
```

2. Filtering Rows Based on Multiple Conditions

```
IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/btaru/Filtering Rows Based on Multiple Conditions.py
Filtered Employees:
  Employee Department  Experience
1      John          IT           7
4      Sora          IT           6
>>>
```

```
Filtering Rows Based on Multiple Conditions.py - C:/Users/btaru/Filtering Rows Based on Multiple Conditions.p...
File Edit Format Run Options Window Help
import pandas as pd

# Sample data
data = {
    "Employee": ["Amy", "John", "Raj", "Kevin", "Sora"],
    "Department": ["HR", "IT", "IT", "Sales", "IT"],
    "Experience": [3, 7, 1, 5, 6]
}

# Create DataFrame
df = pd.DataFrame(data)

# Filter employees in IT with experience greater than 5 years
filtered_df = df[(df["Department"] == "IT") & (df["Experience"] > 5)]

# Display result
print("Filtered Employees:")
print(filtered_df)
```


3. Aggregating and Grouping Data

Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
= RESTART: C:/Users/btaru/Aggregating and Grouping Data.py
Store Summary:
 Store Sales Employees
0 North 2100 5.0
1 South 3200 7.5
2 West 2400 4.0

>>>

Aggregating and Grouping Data.py - C:/Users/btaru/Aggregating and Grouping Data.py (3.11.9)
File Edit Format Run Options Window Help

import pandas as pd

Sample data
data = {
 "Store": ["North", "South", "North", "West", "South", "West"],
 "Sales": [1200, 1500, 900, 1100, 1700, 1300],
 "Employees": [5, 7, 5, 4, 8, 4]
}

Create DataFrame
df = pd.DataFrame(data)

Group by Store
summary = df.groupby("Store").agg({
 "Sales": "sum",
 "Employees": "mean"
}).reset_index()

Display summary
print("Store Summary:")
print(summary)

4. Python Program Using describe()

IDLE Shell 3.11.9

File Edit Shell Debug Options Window Help

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
```

>>>

```
= RESTART: C:/Users/btaru/Python Program Using describe().py
```

```
----- Numeric Summary -----
```

	Age	Salary
count	5.000000	5.000000
mean	35.000000	77000.000000
std	7.905694	23874.672773
min	25.000000	50000.000000
25%	30.000000	60000.000000
50%	35.000000	75000.000000
75%	40.000000	90000.000000
max	45.000000	110000.000000

```
----- Categorical Summary -----
```

	Department
count	5
unique	3
top	HR
freq	2

>>>

|

Python Program Using describe().py - C:/Users/btaru/Python Program Using describe().py (3.11.9)

File Edit Format Run Options Window Help

```
import pandas as pd
```

```
# Sample data
```

```
data = {
    "Age": [25, 30, 35, 40, 45],
    "Salary": [50000, 60000, 75000, 90000, 110000],
    "Department": ["HR", "IT", "IT", "Marketing", "HR"]
}
```

```
# Create DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Numeric summary
```

```
print("----- Numeric Summary -----")
```

```
print(df.describe())
```

```
# Categorical summary
```

```
print("\n----- Categorical Summary -----")
```

```
print(df.describe(include="str"))
```


5. Python Program Using head() and tail()

Handling Missing Data.py - C:/Users/btaru/Handling Missing Data.py (3.11.9)

File Edit Format Run Options Window Help

```
import pandas as pd
import numpy as np

# Sample data with missing values
data = {
    "Name": ["Alice", "Bob", "Charlie", "David"],
    "Age": [25, np.nan, 30, 22],
    "Salary": [50000, 60000, np.nan, 45000]
}

# Create DataFrame
df = pd.DataFrame(data)

# Check missing values
print("Missing Values:")
print(df.isnull().sum())

# Fill missing Age with median
df["Age"] = df["Age"].fillna(df["Age"].median())

# Drop rows where Salary is missing
df = df.dropna(subset=["Salary"])

# Display cleaned DataFrame
print("\nCleaned DataFrame:")
print(df)
```

IDLE Shell 3.11.9

File Edit Shell Debug Options Window Help

Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>

= RESTART: C:/Users/btaru/Handling Missing Data.py

Missing Values:

Name 0

Age 1

Salary 1

dtype: int64

Cleaned DataFrame:

	Name	Age	Salary
--	------	-----	--------

0	Alice	25.0	50000.0
---	-------	------	---------

1	Bob	25.0	60000.0
---	-----	------	---------

3	David	22.0	45000.0
---	-------	------	---------

>>>